```
-- Extensão usada para geração de UUIDs no PostgreSQL
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Tipo enumerado para definir o papel do usuário no sistema
CREATE TYPE user_role AS ENUM ('user', 'admin');

-- Tabela de usuários do sistema
-- Armazena dados básicos de autenticação e autorização
CREATE TABLE users (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    role user_role NOT NULL DEFAULT 'user'
);

-- Tabela de produtos disponíveis na loja
-- Controla estoque, preço e informações visuais
CREATE TABLE products (
    id UUID PRIMARY KEY,
    name TEXT NOT NULL,
    price NUMERIC(10, 2) NOT NULL CHECK (price > 0),
    description TEXT NOT NULL,
    stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0),
    url_img TEXT NOT NULL
        DEFAULT 'https://pub-
fa9024cfefc44645b919b992e1a15089.r2.dev/default.jpg'
);

-- Itens no carrinho de compras do usuário
-- Cada usuário só pode ter um registro por produto
CREATE TABLE cart_itens (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    product_id UUID NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),

    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
    UNIQUE (user_id, product_id)
);

-- Cupons de desconto aplicáveis em pedidos
CREATE TABLE coupons (
    id UUID PRIMARY KEY,
    code TEXT NOT NULL UNIQUE,
    discount_percentage INTEGER NOT NULL CHECK (discount_percentage > 0
AND discount_percentage <= 100),
    expires_at TIMESTAMP NOT NULL
);

-- Pedidos realizados pelos usuários
CREATE TABLE orders (
    id UUID PRIMARY KEY,
```

```
    user_id UUID NOT NULL,
    coupon_id UUID,
    purchase_at TIMESTAMP NOT NULL,

    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (coupon_id) REFERENCES coupons(id)
);

-- Registro “congelado” dos produtos no momento da compra
-- Garante que o histórico do pedido não mude se o produto for alterado
depois
CREATE TABLE product_register (
    id UUID PRIMARY KEY,
    order_id UUID NOT NULL,
    product_id UUID NOT NULL,
    name TEXT NOT NULL,
    price NUMERIC(10, 2) NOT NULL CHECK (price > 0),
    description TEXT NOT NULL,
    url_img TEXT NOT NULL
        DEFAULT 'https://pub-
fa9024cfefc44645b919b992e1a15089.r2.dev/default.jpg',

    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE
);

-- Relação entre pedido e produtos comprados
-- Permite armazenar quantidade de cada item no pedido
CREATE TABLE product_order (
    id UUID PRIMARY KEY,
    order_id UUID NOT NULL,
    product_register_id UUID NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),

    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (product_register_id) REFERENCES product_register(id)
);

-- Avaliações feitas pelos usuários nos produtos
-- Cada usuário só pode avaliar um produto uma única vez
CREATE TABLE review (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    product_id UUID NOT NULL,
    rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comment TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL,

    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE,
    UNIQUE (user_id, product_id)
);


-- Trigger para INSERT no carrinho:
```

```
-- Se o produto já existir no carrinho do usuário,
-- soma a quantidade ao invés de criar outra linha

CREATE OR REPLACE FUNCTION cart_itens_upsert_quantity()
RETURNS TRIGGER AS $$
BEGIN
  IF EXISTS (
    SELECT 1
    FROM cart_itens
    WHERE user_id = NEW.user_id
      AND product_id = NEW.product_id
  ) THEN
    UPDATE cart_itens
    SET quantity = quantity + NEW.quantity
    WHERE user_id = NEW.user_id
      AND product_id = NEW.product_id;

    RETURN NULL;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_cart_itens_upsert ON cart_itens;

CREATE TRIGGER trg_cart_itens_upsert
BEFORE INSERT ON cart_itens
FOR EACH ROW
EXECUTE FUNCTION cart_itens_upsert_quantity();


-- Trigger para DELETE no carrinho:
-- Se a quantidade for maior que 1, apenas decrementa
-- Se for igual a 1, permite remover o item

CREATE OR REPLACE FUNCTION cart_itens_decrement_or_delete()
RETURNS TRIGGER AS $$
BEGIN
  IF OLD.quantity > 1 THEN
    UPDATE cart_itens
    SET quantity = quantity - 1
    WHERE id = OLD.id;

    RETURN NULL;
  END IF;

  RETURN OLD;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_cart_itens_decrement ON cart_itens;

CREATE TRIGGER trg_cart_itens_decrement
```

```
BEFORE DELETE ON cart_itens
FOR EACH ROW
EXECUTE FUNCTION cart_itens_decrement_or_delete();


-- Trigger em product_order:
-- Sempre que um item é adicionado a um pedido,
-- o estoque do produto original é reduzido automaticamente


CREATE OR REPLACE FUNCTION decrease_product_stock()
RETURNS TRIGGER AS $$
DECLARE
    v_product_id UUID;
BEGIN
    SELECT product_id
    INTO v_product_id
    FROM product_register
    WHERE id = NEW.product_register_id;

    UPDATE products
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE id = v_product_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Produto não encontrado para o product_register
%', NEW.product_register_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_decrease_product_stock ON product_order;

CREATE TRIGGER trg_decrease_product_stock
AFTER INSERT ON product_order
FOR EACH ROW
EXECUTE FUNCTION decrease_product_stock();


-- View que detalha os pedidos
-- Motivo: facilitar consultas que precisam listar os produtos de cada
pedido,
-- evitando JOINs repetidos no código da aplicação.

CREATE OR REPLACE VIEW vw_order_details AS
SELECT
    o.id AS order_id,
    o.user_id,
    o.coupon_id,
    o.purchase_at AS order_purchase_at,

    pr.id AS product_id,
```

```
    pr.name AS product_name,
    pr.price AS product_price,
    pr.description AS product_description,
    pr.url_img AS product_url_img,

    po.quantity AS product_quantity

FROM orders o
JOIN product_order po
    ON o.id = po.order_id
JOIN product_register pr
    ON po.product_register_id = pr.id;



-- View que resume os pedidos
-- Motivo: fornecer uma visão agregada de cada pedido (total de produtos,
-- itens e valor), simplificando relatórios e telas de histórico de
compras.

CREATE OR REPLACE VIEW vw_order_summary AS
SELECT
    o.id AS order_id,
    o.user_id,
    o.coupon_id,
    o.purchase_at AS order_purchase_at,

    COUNT(po.product_register_id)            AS total_products,
    SUM(po.quantity)                         AS total_items,
    SUM(pr.price * po.quantity)              AS total_value

FROM orders o
JOIN product_order po
    ON o.id = po.order_id
JOIN product_register pr
    ON po.product_register_id = pr.id
GROUP BY
    o.id,
    o.user_id,
    o.coupon_id,
    o.purchase_at;



-- Querys usando o pscopg2

    -- Querys do users

        INSERT INTO users (id, name, email, password)
        VALUES (%s, %s, %s, %s);

        SELECT id, name, email, password, role
        FROM users
        WHERE id = %s;
```

```
    SELECT id, name, email, password, role
    FROM users
    WHERE email = %s;

    UPDATE users
    SET name = %s
    WHERE id = %s
    RETURNING id, name, email, password, role;

    UPDATE users
    SET email = %s
    WHERE id = %s
    RETURNING id, name, email, password, role;

    UPDATE users
    SET password = %s
    WHERE id = %s
    RETURNING id, name, email, password, role;

    DELETE FROM users
    WHERE id = %s;


-- Querys do products

    SELECT
        id,
        name,
        price,
        description,
        stock_quantity,
        url_img
    FROM products;

    SELECT
        id,
        name,
        price,
        description,
        stock_quantity,
        url_img
    FROM products
    WHERE id = %s;

    SELECT
        id,
        name,
        price,
        description,
        stock_quantity,
        url_img
    FROM products
    WHERE price <= %s;
```

```
-- Querys do cart_itens

    INSERT INTO cart_itens (id, user_id, product_id, quantity)
    VALUES (%s, %s, %s, %s);

    DELETE FROM cart_itens
    WHERE user_id = %s
        AND product_id = %s;

    SELECT
        p.id,
        p.name,
        p.price,
        p.url_img,
        ci.quantity
    FROM cart_itens ci
    JOIN products p ON p.id = ci.product_id
    WHERE ci.user_id = %s;

    SELECT
        p.id,
        p.name,
        p.price,
        p.url_img,
        ci.quantity
    FROM cart_itens ci
    JOIN products p ON p.id = ci.product_id
    WHERE ci.user_id = %s AND ci.product_id = %s;

    DELETE FROM cart_itens
    WHERE user_id = %s;

    SELECT
      p.id,
      p.name,
      p.price,
      p.url_img,
      ci.quantity,
      (p.price * ci.quantity) AS item_total,
    SUM(p.price * ci.quantity)
    OVER (PARTITION BY ci.user_id) AS cart_total
    FROM cart_itens ci
    JOIN products p ON p.id = ci.product_id
    WHERE ci.user_id = %s;



-- Querys do coupons

    INSERT INTO coupons
    (id, code, discount_percentage, expires_at)
    VALUES (%s, %s, %s, %s);
```

```
    SELECT
        id,
        code,
        discount_percentage,
        expires_at
    FROM coupons
    WHERE code = %s;

    SELECT
        id,
        code,
        discount_percentage,
        expires_at
    FROM coupons
    WHERE id = %s;

    SELECT
        id,
        code,
        discount_percentage,
        expires_at
    FROM coupons;

    UPDATE coupons
    SET discount_percentage = %s
    WHERE id = %s
    RETURNING id, code, discount_percentage, expires_at;

    DELETE FROM coupons
    WHERE id = %s;



-- Querys do orders

    INSERT INTO orders
    (id, user_id, coupon_id, purchase_at)
    VALUES (%s, %s, %s, %s);

    SELECT
        id,
        user_id,
        coupon_id,
        purchase_at
    FROM orders
    WHERE id = %s;

    SELECT
        order_id,
        user_id,
        coupon_id,
        order_purchase_at,
        product_id,
```

```
        product_name,
        product_price,
        product_description,
        product_url_img,
        product_quantity,
        SUM(product_price * product_quantity) AS purchase_value
    FROM vw_order_details
    WHERE order_id = %s
    GROUP BY
        order_id,
        user_id,
        coupon_id,
        order_purchase_at,
        product_id,
        product_name,
        product_price,
        product_description,
        product_url_img,
        product_quantity;

    DELETE from orders
    WHERE id = %s;

    SELECT
        order_id,
        user_id,
        coupon_id,
        order_purchase_at,
        total_products,
        total_items,
        total_value
    FROM vw_order_summary
    WHERE user_id = %s
    ORDER BY order_purchase_at DESC;



-- Querys do product_register

    INSERT INTO product_register
    (id, order_id, product_id, name, price, description, url_img)
    SELECT
        gen_random_uuid(),
        %s,
        %s,
        p.name,
        p.price,
        p.description,
        p.url_img
    FROM products p
    WHERE p.id = %s
    RETURNING id;
```

```
-- Querys do product_order

    INSERT INTO product_order
    (id, order_id, product_register_id, quantity)
    VALUES (%s, %s, %s, %s);


-- Querys do review

    INSERT INTO review
        (
        id,
        user_id,
        product_id,
        rating,
        comment,
        created_at
        )
    VALUES (%s, %s, %s, %s, %s, %s);

    SELECT
        id,
        user_id,
        product_id,
        rating,
        comment,
        created_at
    FROM review
    WHERE product_id = %s;

    SELECT
        id,
        user_id,
        product_id,
        rating,
        comment,
        created_at
    FROM review
    WHERE user_id = %s;

    SELECT
        id,
        user_id,
        product_id,
        rating,
        comment,
        created_at
    FROM review
    WHERE id = %s;

    SELECT
        id,
        user_id,
        product_id,
```

```
            rating,
            comment,
            created_at
        FROM review
        WHERE user_id = %s
            AND product_id = %s;

        UPDATE review
        SET comment = %s
        WHERE id = %s;

        UPDATE review
        SET rating = %s
        WHERE id = %s;

        DELETE FROM review
        WHERE id = %s;


-- CONSULTAS AVANÇADAS

    -- 1) Ranking de produtos mais vendidos
    -- Mostra os produtos ordenados pela quantidade total vendida
    SELECT
        pr.product_id,
        pr.name,
        SUM(po.quantity) AS total_sold
    FROM product_order po
    JOIN product_register pr
        ON po.product_register_id = pr.id
    GROUP BY pr.product_id, pr.name
    ORDER BY total_sold DESC;

    -- 2) Usuários que mais compraram (em valor)
    -- Soma o valor total gasto por cada usuário
    SELECT
        o.user_id,
        u.name,
        SUM(pr.price * po.quantity) AS total_spent
    FROM orders o
    JOIN users u
        ON u.id = o.user_id
    JOIN product_order po
        ON po.order_id = o.id
    JOIN product_register pr
        ON pr.id = po.product_register_id
    GROUP BY o.user_id, u.name
    ORDER BY total_spent DESC;

    -- 3) Produtos com média de avaliação
    -- Exibe cada produto com sua nota média
    SELECT
        p.id,
        p.name,
```

```
    ROUND(AVG(r.rating), 2) AS avg_rating,
    COUNT(r.id) AS total_reviews
FROM products p
LEFT JOIN review r
    ON r.product_id = p.id
GROUP BY p.id, p.name
ORDER BY avg_rating DESC NULLS LAST;

-- 4) Pedidos acima de um determinado valor
-- Útil para relatórios gerenciais
SELECT
    order_id,
    user_id,
    total_items,
    total_value
FROM vw_order_summary
WHERE total_value >= %s
ORDER BY total_value DESC;

-- 5) Produtos com estoque baixo
-- Ajuda a identificar itens que precisam ser repostos
SELECT
    id,
    name,
    stock_quantity
FROM products
WHERE stock_quantity <= %s
ORDER BY stock_quantity ASC;
```